Graphic Era deemed to be University
DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

(Try to choose a catchy title. Max 20 words).

AMBULANCE AND EMERGENCY DISPATCH SYSTEM

Team Name: *Team #*	*The Apex Achievers*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Hajariwala, Neel – 2510011575* *neelhajariwala60@gmail.com* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Namdev, Keshav – 2510360097* *namdevkeshav2212@gmail.com* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Chauhan, Lichhavi – 2510370231* *lichhavichauhan@gmail.com* 
Team member 4 *(Last Name, name: student ID, email, picture):*	*Ahlawat, Aditya – 2512510001* *adi.14ah@gmail.com* 

PHASE-III PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract (2 pts)

(Brief restatement of your project's main goal. Max 300 words).

1. We have successfully wrapped up the core C++ logic for our Ambulance and Emergency Dispatch System. This includes setting up emergency priority handling, picking the right ambulance and hospital, and figuring out the shortest routes using Dijkstra's algorithm. Everything runs smoothly on core data structures to keep emergency requests organized and ensure fast resource allocation.
2. Right now, our main goal is taking those solid C++ backend modules and hooking them up with a full desktop interface using Qt Widgets. We're actively building out the desktop screens, tying them directly into our existing backend code, and running thorough testing. This transition lets us show off how data structures and algorithms actually come together in a real, functional desktop application built from scratch.

Updated Project Approach and Architecture (2 pts)

(Describe your current approach, including system design, communication protocols, libraries used, etc. Max 300 words).

1. We designed this project using a clean, modular layout in C++, specifically tailored for Windows desktop usage. For the graphical interface, we are relying on Qt Widgets, which gives us all the native windows, buttons, tables, and input forms we need to make it feel responsive and easy to use.
2. Under the hood, the frontend connects seamlessly with our core C++ logic handling emergency dispatch, vehicle routing, and hospital matching. We used a Max-Heap Priority Queue so that critical emergency cases instantly jump to the top of the queue, while Dijkstra's algorithm handles the navigation math. To keep things lightweight and reliable, we're storing logs and dispatch records using CSV files. The layout features dedicated screens customized specifically for administrators and ambulance drivers.

Tasks Completed (7 pts)

(Describe the main tasks that have been assigned and already completed. Max 250 words).

Task Completed	Team Member
1. C++ backend development and core logic setup.	1. Keshav Namdev
2. Emergency priority queue implementation using max-heap.	2. Neel Hajariwala
3. Dijkstra's algorithm integration and ambulance selection logic.	3. Keshav Namdev
4. Core project architecture and coding.	4. Lichhavi Chauhan
5. Desktop dashboard design and initial documentation.	5. Lichhavi Chauhan
6. Managing hospital records and tracking ambulance status.	6. Aditya Ahlawat
7. System testing, debugging, and error fixing.	7. Aditya Ahlawat / Neel Hajariwala

Challenges/Roadblocks (7 pts)

(Describe the challenges that you have faced or are facing so far and how you plan to solve them. Max 300 words).

1. Shifting our initial workflow over to a dedicated Windows desktop application while making sure all existing C++ backend pieces stayed fully functional.
2. Getting up to speed with Qt Widgets to design a clean, responsive graphical interface entirely through C++.
3. Bridging the gap between the GUI and our core backend modules so data flows smoothly without lag or missing triggers.
4. Setting up strict checks to ensure the same ambulance never gets double-assigned to multiple active emergencies by mistake.
5. Tuning Dijkstra's algorithm so it calculates optimal routes accurately across our map data nodes.
6. Building a smart hospital selection filter that weighs distance, available beds, and specialized medical facilities all at once.
7. Sorting out high-pressure emergency requests effectively based on their true severity and urgency levels.

Project Outcome/Deliverables (2 pts)

(Describe what are the key outcomes / deliverables of the project. Max 200 words).

Our primary outcome is a fully working Ambulance and Emergency Dispatch System powered by C++ and core data structures. The system handles emergency calls effortlessly, sorts critical cases using a Max-Heap Priority Queue, finds the closest available ambulance, and runs Dijkstra's algorithm for the quickest routes. It also filters hospitals based on distance and bed availability to prevent duplicate dispatches.
We are packaging this into a native Windows desktop app using Qt Widgets, giving dispatchers and drivers separate, tailored dashboards for managing calls, checking vehicle status, and reviewing history logs via clean CSV file storage.
The final product brings our core C++ logic together with an intuitive desktop interface, showing firsthand how data structures solve practical, real-world emergency management problems.

Progress Overview (2 pts)

(Summarize how much of the project is done, what's behind schedule, what's ahead of schedule. Max 200 words.)

All the heavy lifting on the C++ backend side—like priority queues, routing math, and resource matching—is fully done and individually tested.
We originally looked at a web-based layout, but after updating our goals, we've successfully pivoted to building a robust Windows desktop application using C++ and Qt Widgets.
Right now, we are right in the middle of hooking up the GUI to our backend logic. The login screens and dashboards are taking shape, and what's left is tying everything together, running end-to-end tests, and compiling the final executable.
Our main focus right now is making sure our custom data structures plug into the new desktop environment smoothly without any hiccups.

Codebase Information (2 pts)

(Repository link, branch, and information about important commits.)

1. We are keeping all our source code neat and organized on GitHub. The repository houses our C++ backend code, modular files, UI assets, and project logs. It makes it super easy for all of us to collaborate, pull updates, and track changes in real time as we code.
2. GitHub Repository: https://github.com/LICHHAVI26/DSCPP-PBL

You can check out everyone's individual contributions, feature branches, and commit histories directly through the repository link above.

Testing and Validation Status (2 pts)

(Provide information about any tests conducted)

Test Type	Status (Pass/Fail)	Notes
1. Core functionality testing	PASS	All main features and backend logic are running smoothly.
2. System integration testing	PASS	GUI and backend data flow working well together.

Deliverables Progress (2 pts)

(Summarize the current status of all key project deliverables mentioned earlier. Indicate whether each deliverable is completed, in progress, or pending.)

1. Completed: Built out the full C++ backend, including emergency queues, heap logic, route calculation, hospital filtering, and CSV record tracking.
2. In Progress: Shifting the app into a Windows desktop format using Qt Widgets and building out the driver and admin interfaces.
3. Pending: Final integration between the GUI and backend logic, full system testing, bug fixes, and compiling the final Windows executable.

Everything is on track to deliver a solid, fully functional desktop application showcasing practical data structures and algorithms.